

Reporte de Gramática

Forma de Backus-Naur (BNF)

Daniel Alonso Gracia Pinto

2026-05-05 16:03:17

1. Reglas de Producción y Conjuntos de Predicción

No terminal	Producción	Pred. Set
<code><start></code>	<code>::= <global_code> EOF</code>	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
<code><global_code></code>	<code>::= <g_stmt> <global_code></code> <code> ε</code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero} {EOF}
<code><block_code></code>	<code>::= <b_stmt> <block_code></code> <code> ε</code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero} {CLOSING_KEY, caso, porDefecto}
<code><g_stmt></code>	<code>::= <decl_keyword> <g_decl_list></code> <code> <g_semicolon_opt></code> <code> si OPENING_PAR <expr> CLOSING_PAR</code> <code><block_stmt> <g_sino_opt></code> <code> mientras OPENING_PAR <expr> CLOSING_PAR</code> <code><block_stmt></code> <code> hacer <block_stmt> mientras OPENING_PAR</code> <code><expr> CLOSING_PAR <g_semicolon_opt></code> <code> para OPENING_PAR <para_init> SEMICOLON</code> <code><expr_opt> SEMICOLON <expr_opt> CLOSING_PAR</code> <code><block_stmt></code> <code> elegir OPENING_PAR <expr> CLOSING_PAR</code> <code>OPENING_KEY <casos_list> CLOSING_KEY</code> <code> romper <g_semicolon_opt></code> <code> continuar <g_semicolon_opt></code> <code> intentar <block_stmt> capturar <block_stmt></code> <code> funcion IDENT OPENING_PAR <params_opt></code> <code>CLOSING_PAR <block_stmt></code> <code> <block_stmt></code> <code> retornar <expr_opt> <g_semicolon_opt></code> <code> <consola_call> <g_semicolon_opt></code> <code> <expr> <g_semicolon_opt></code> <code> SEMICOLON</code>	{const, mut, var} {si} {mientras} {hacer} {para} {elegir} {romper} {continuar} {intentar} {funcion} {OPENING_KEY} {retornar} {consola} {Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero} {SEMICOLON}
<code><g_decl_list></code>	<code>::= IDENT <g_decl_init> <g_decl_list_prime></code>	{IDENT}
<code><g_decl_init></code>	<code>::= ASSIGN <expr_or_crear></code>	{ASSIGN}

No terminal	Producción	Pred. Set
	ϵ	{Arreglo, Booleano, COMMA, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle g_decl_list_prime \rangle$	::= COMMA $\langle g_decl_list \rangle$	{COMMA}
	ϵ	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle g_semicolon_opt \rangle$	::= SEMICOLON	{SEMICOLON}
	ϵ	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle g_sino_opt \rangle$	::= sino $\langle g_sino_tail \rangle$	{sino}
	ϵ	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle g_sino_tail \rangle$	::= si OPENING_PAR $\langle expr \rangle$ CLOSING_PAR $\langle block_stmt \rangle$ $\langle g_sino_opt \rangle$	{si}
	$\langle block_stmt \rangle$	{OPENING_KEY}
$\langle b_stmt \rangle$	::= $\langle decl_keyword \rangle$ $\langle b_decl_list \rangle$	{const, mut, var}
	$\langle b_semicolon_opt \rangle$	
	si OPENING_PAR $\langle expr \rangle$ CLOSING_PAR $\langle block_stmt \rangle$ $\langle b_sino_opt \rangle$	{si}
	mientras OPENING_PAR $\langle expr \rangle$ CLOSING_PAR $\langle block_stmt \rangle$	{mientras}
	hacer $\langle block_stmt \rangle$ mientras OPENING_PAR $\langle expr \rangle$ CLOSING_PAR $\langle b_semicolon_opt \rangle$	{hacer}
	para OPENING_PAR $\langle para_init \rangle$ SEMICOLON $\langle expr_opt \rangle$ SEMICOLON $\langle expr_opt \rangle$ CLOSING_PAR $\langle block_stmt \rangle$	{para}
	elegir OPENING_PAR $\langle expr \rangle$ CLOSING_PAR OPENING_KEY $\langle casos_list \rangle$ CLOSING_KEY	{elegir}
	romper $\langle b_semicolon_opt \rangle$	{romper}
	continuar $\langle b_semicolon_opt \rangle$	{continuar}
	intentar $\langle block_stmt \rangle$ capturar $\langle block_stmt \rangle$	{intentar}
	funcion IDENT OPENING_PAR $\langle params_opt \rangle$ CLOSING_PAR $\langle block_stmt \rangle$	{funcion}
	$\langle block_stmt \rangle$	{OPENING_KEY}
	retornar $\langle expr_opt \rangle$ $\langle b_semicolon_opt \rangle$	{retornar}
	$\langle consola_call \rangle$ $\langle b_semicolon_opt \rangle$	{consola}
	$\langle expr \rangle$ $\langle b_semicolon_opt \rangle$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
	SEMICOLON	{SEMICOLON}
$\langle b_decl_list \rangle$	::= IDENT $\langle b_decl_init \rangle$ $\langle b_decl_list_prime \rangle$	{IDENT}
$\langle b_decl_init \rangle$	::= ASSIGN $\langle expr_or_crear \rangle$	{ASSIGN}

No terminal	Producción	Pred. Set
	ϵ	{Arreglo, Booleano, CLOSING_KEY, COMMA, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
$\langle b_decl_list_prime \rangle$	::= COMMA $\langle b_decl_list \rangle$ ϵ	{COMMA} {Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
$\langle b_semicolon_opt \rangle$	::= SEMICOLON ϵ	{SEMICOLON} {Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
$\langle b_sino_opt \rangle$	::= sino $\langle b_sino_tail \rangle$ ϵ	{sino} {Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
$\langle b_sino_tail \rangle$	::= si OPENING_PAR $\langle expr \rangle$ CLOSING_PAR $\langle block_stmt \rangle$ $\langle b_sino_opt \rangle$ $\langle block_stmt \rangle$	{si} {OPENING_KEY}
$\langle decl_keyword \rangle$	::= var mut const	{var} {mut} {const}
$\langle expr_or_crear \rangle$	::= crear $\langle crear_type \rangle$ $\langle crear_args_opt \rangle$ $\langle expr \rangle$	{crear} {Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
$\langle crear_args_opt \rangle$	::= OPENING_PAR $\langle args_opt \rangle$ CLOSING_PAR ϵ	{OPENING_PAR} {AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}

No terminal	Producción	Pred. Set
<crear_type>	::= IDENT Arreglo Cadena Matriz	{IDENT} {Arreglo} {Cadena} {Matriz}
<para_init>	::= <decl_keyword> {b_decl_list} <expr_opt>	{const, mut, var} {Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, falso, indefinido, nulo, verdadero}
<casos_list>	::= caso <expr> COLON <block_code> {casos_list} porDefecto COLON <block_code> ε	{caso} {porDefecto} {CLOSING_KEY}
<block_stmt>	::= OPENING_KEY <block_code> CLOSING_KEY	{OPENING_KEY}
<expr_opt>	::= <expr> ε	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero} {Arreglo, Booleano, CLOSING_KEY, CLOSING_PAR, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<expr>	::= <factor> <expr_prime>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
<expr_prime>	::= PLUS <factor> <expr_prime> MINUS <factor> <expr_prime> TIMES <factor> <expr_prime> DIV <factor> <expr_prime> MOD <factor> <expr_prime> POWER <factor> <expr_prime> EQUAL <factor> <expr_prime> STRICT_EQUAL <factor> <expr_prime> NEQ <factor> <expr_prime> STRICT_NEQ <factor> <expr_prime> LESS <factor> <expr_prime> GREATER <factor> <expr_prime> LEQ <factor> <expr_prime> GEQ <factor> <expr_prime> AND <factor> <expr_prime> OR <factor> <expr_prime> TERNARY <ternary_mid_expr> COLON <expr_or_consola> ε	{PLUS} {MINUS} {TIMES} {DIV} {MOD} {POWER} {EQUAL} {STRICT_EQUAL} {NEQ} {STRICT_NEQ} {LESS} {GREATER} {LEQ} {GEQ} {AND} {OR} {TERNARY} {AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<expr_or_consola>	::= <expr>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}

No terminal	Producción	Pred. Set
	<consola_call>	{consola}
<arg_expr>	::= <factor> <arg_expr_prime>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
<arg_expr_prime>	::= PLUS <factor> <arg_expr_prime> MINUS <factor> <arg_expr_prime> TIMES <factor> <arg_expr_prime> DIV <factor> <arg_expr_prime> MOD <factor> <arg_expr_prime> POWER <factor> <arg_expr_prime> EQUAL <factor> <arg_expr_prime> STRICT_EQUAL <factor> <arg_expr_prime> NEQ <factor> <arg_expr_prime> STRICT_NEQ <factor> <arg_expr_prime> LESS <factor> <arg_expr_prime> GREATER <factor> <arg_expr_prime> LEQ <factor> <arg_expr_prime> GEQ <factor> <arg_expr_prime> AND <factor> <arg_expr_prime> OR <factor> <arg_expr_prime> TERNARY <ternary_mid_expr> COLON <arg_expr_or_consola> ε	{PLUS} {MINUS} {TIMES} {DIV} {MOD} {POWER} {EQUAL} {STRICT_EQUAL} {NEQ} {STRICT_NEQ} {LESS} {GREATER} {LEQ} {GEQ} {AND} {OR} {TERNARY} {CLOSING_PAR, COMMA}
<arg_expr_or_consola>	::= <arg_expr> <consola_call>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero} {consola}
<ternary_mid_expr>	::= <factor> <ternary_mid_expr_prime> <consola_call>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero} {consola}
<ternary_mid_expr_prime>	::= PLUS <factor> <ternary_mid_expr_prime> MINUS <factor> <ternary_mid_expr_prime> TIMES <factor> <ternary_mid_expr_prime> DIV <factor> <ternary_mid_expr_prime> MOD <factor> <ternary_mid_expr_prime> POWER <factor> <ternary_mid_expr_prime> EQUAL <factor> <ternary_mid_expr_prime> STRICT_EQUAL <factor> <ternary_mid_expr_prime> NEQ <factor> <ternary_mid_expr_prime> STRICT_NEQ <factor> <ternary_mid_expr_prime> LESS <factor> <ternary_mid_expr_prime> GREATER <factor> <ternary_mid_expr_prime> LEQ <factor> <ternary_mid_expr_prime> GEQ <factor> <ternary_mid_expr_prime> AND <factor> <ternary_mid_expr_prime> OR <factor> <ternary_mid_expr_prime> TERNARY <ternary_mid_expr> COLON <ternary_mid_expr> ε	{PLUS} {MINUS} {TIMES} {DIV} {MOD} {POWER} {EQUAL} {STRICT_EQUAL} {NEQ} {STRICT_NEQ} {LESS} {GREATER} {LEQ} {GEQ} {AND} {OR} {TERNARY} {COLON}
<consola_call>	::= consola PERIOD <consola_method> OPENING_PAR <args_opt> CLOSING_PAR	{consola}
<consola_method>	::= afirmar agrupar error escribir info	{afirmar} {agrupar} {error} {escribir} {info}

No terminal	Producción	Pred. Set
	limpiar	{limpiar}
	tabla	{tabla}
$\langle \text{factor} \rangle$	::= IDENT $\langle \text{factor_tail} \rangle$ Mate $\langle \text{factor_tail} \rangle$ Numero $\langle \text{factor_tail} \rangle$ Arreglo $\langle \text{factor_tail} \rangle$ Cadena $\langle \text{factor_tail} \rangle$ Matriz $\langle \text{factor_tail} \rangle$ Booleano $\langle \text{factor_tail} \rangle$ OPENING_PAR $\langle \text{args_opt} \rangle$ CLOSING_PAR $\langle \text{factor_tail} \rangle$ OPENING_BRA $\langle \text{array_args_opt} \rangle$ CLOSING_BRA $\langle \text{factor_tail} \rangle$ OPENING_KEY $\langle \text{obj_elements} \rangle$ CLOSING_KEY NUMBER STR verdadero falso nulo indefinido Infinito NuN MINUS $\langle \text{factor} \rangle$ PLUS $\langle \text{factor} \rangle$ NOT $\langle \text{factor} \rangle$	{IDENT} {Mate} {Numero} {Arreglo} {Cadena} {Matriz} {Booleano} {OPENING_PAR} {OPENING_BRA} {OPENING_KEY} {NUMBER} {STR} {verdadero} {falso} {nulo} {indefinido} {Infinito} {NuN} {MINUS} {PLUS} {NOT}
$\langle \text{factor_tail} \rangle$	::= OPENING_PAR $\langle \text{args_opt} \rangle$ CLOSING_PAR $\langle \text{factor_tail} \rangle$ OPENING_BRA $\langle \text{expr} \rangle$ CLOSING_BRA $\langle \text{factor_tail} \rangle$ PERIOD IDENT $\langle \text{factor_tail} \rangle$ ASSIGN $\langle \text{expr_or_crear} \rangle$ PLUS_ASSIGN $\langle \text{expr} \rangle$ MINUS_ASSIGN $\langle \text{expr} \rangle$ TIMES_ASSIGN $\langle \text{expr} \rangle$ DIV_ASSIGN $\langle \text{expr} \rangle$ MOD_ASSIGN $\langle \text{expr} \rangle$ POWER_ASSIGN $\langle \text{expr} \rangle$ ARROW $\langle \text{arrow_body} \rangle$ INCREMENT $\langle \text{factor_tail} \rangle$ DECREMENT $\langle \text{factor_tail} \rangle$ ϵ	{OPENING_PAR} {OPENING_BRA} {PERIOD} {ASSIGN} {PLUS_ASSIGN} {MINUS_ASSIGN} {TIMES_ASSIGN} {DIV_ASSIGN} {MOD_ASSIGN} {POWER_ASSIGN} {ARROW} {INCREMENT} {DECREMENT} {AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
$\langle \text{arrow_body} \rangle$	::= $\langle \text{block_stmt} \rangle$ $\langle \text{expr} \rangle$	{OPENING_KEY} {Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
$\langle \text{params_opt} \rangle$	::= IDENT $\langle \text{params_prime} \rangle$ ϵ	{IDENT} {CLOSING_PAR}
$\langle \text{params_prime} \rangle$	::= COMMA IDENT $\langle \text{params_prime} \rangle$ ϵ	{COMMA} {CLOSING_PAR}

No terminal	Producción	Pred. Set
$\langle \text{args_opt} \rangle$	$::= \langle \text{arg_expr} \rangle \langle \text{args_prime} \rangle$ $ \varepsilon$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero} {CLOSING_PAR}
$\langle \text{args_prime} \rangle$	$::= \text{COMMA} \langle \text{arg_expr} \rangle \langle \text{args_prime} \rangle$ $ \varepsilon$	{COMMA} {CLOSING_PAR}
$\langle \text{array_args_opt} \rangle$	$::= \langle \text{expr} \rangle \langle \text{array_args_prime} \rangle$ $ \varepsilon$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero} {CLOSING_BRA}
$\langle \text{array_args_prime} \rangle$	$::= \text{COMMA} \langle \text{expr} \rangle \langle \text{array_args_prime} \rangle$ $ \varepsilon$	{COMMA} {CLOSING_BRA}
$\langle \text{obj_elements} \rangle$	$::= \text{IDENT} \langle \text{obj_tail} \rangle \langle \text{obj_prime} \rangle$ $ \varepsilon$	{IDENT} {CLOSING_KEY}
$\langle \text{obj_tail} \rangle$	$::= \text{COLON} \langle \text{expr} \rangle$ $ \text{OPENING_PAR} \langle \text{params_opt} \rangle \text{CLOSING_PAR}$ $\langle \text{block_stmt} \rangle$	{COLON} {OPENING_PAR}
$\langle \text{obj_prime} \rangle$	$::= \text{COMMA} \text{IDENT} \langle \text{obj_tail} \rangle \langle \text{obj_prime} \rangle$ $ \varepsilon$	{COMMA} {CLOSING_KEY}

2. Predicción Total por No Terminal

No terminal	Pred. Set total
$\langle \text{start} \rangle$	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{global_code} \rangle$	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{block_code} \rangle$	{Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
$\langle \text{g_stmt} \rangle$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{g_decl_list} \rangle$	{IDENT}
$\langle \text{g_decl_init} \rangle$	{ASSIGN, Arreglo, Booleano, COMMA, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{g_decl_list_prime} \rangle$	{Arreglo, Booleano, COMMA, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{g_semicolon_opt} \rangle$	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{g_sino_opt} \rangle$	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, sino, var, verdadero}
$\langle \text{g_sino_tail} \rangle$	{OPENING_KEY, si}

No terminal	Pred. Set total
<b_stmt>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
<b_decl_list>	{IDENT}
<b_decl_init>	{ASSIGN, Arreglo, Booleano, CLOSING_KEY, COMMA, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<b_decl_list_prime>	{Arreglo, Booleano, CLOSING_KEY, COMMA, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<b_semicolon_opt>	{Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<b_sino_opt>	{Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, sino, var, verdadero}
<b_sino_tail>	{OPENING_KEY, si}
<decl_keyword>	{const, mut, var}
<expr_or_crear>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, crear, falso, indefinido, nulo, verdadero}
<crear_args_opt>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<crear_type>	{Arreglo, Cadena, IDENT, Matriz}
<para_init>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, const, falso, indefinido, mut, nulo, var, verdadero}
<casos_list>	{CLOSING_KEY, caso, porDefecto}
<block_stmt>	{OPENING_KEY}
<expr_opt>	{Arreglo, Booleano, CLOSING_KEY, CLOSING_PAR, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<expr>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
<expr_prime>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<expr_or_consola>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, consola, falso, indefinido, nulo, verdadero}
<arg_expr>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
<arg_expr_prime>	{AND, CLOSING_PAR, COMMA, DIV, EQUAL, GEQ, GREATER, LEQ, LESS, MINUS, MOD, NEQ, OR, PLUS, POWER, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES}
<arg_expr_or_consola>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, consola, falso, indefinido, nulo, verdadero}
<ternary_mid_expr>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, consola, falso, indefinido, nulo, verdadero}
<ternary_mid_expr_prime>	{AND, COLON, DIV, EQUAL, GEQ, GREATER, LEQ, LESS, MINUS, MOD, NEQ, OR, PLUS, POWER, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES}
<consola_call>	{consola}
<consola_method>	{afirmar, agrupar, error, escribir, info, limpiar, tabla}

No terminal	Pred. Set total
$\langle \text{factor} \rangle$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
$\langle \text{factor_tail} \rangle$	{AND, ARROW, ASSIGN, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DECREMENT, DIV, DIV_ASSIGN, EOF, EQUAL, GEQ, GREATER, IDENT, INCREMENT, Infinito, LEQ, LESS, MINUS, MINUS_ASSIGN, MOD, MOD_ASSIGN, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PERIOD, PLUS, PLUS_ASSIGN, POWER, POWER_ASSIGN, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, TIMES_ASSIGN, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
$\langle \text{arrow_body} \rangle$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
$\langle \text{params_opt} \rangle$	{CLOSING_PAR, IDENT}
$\langle \text{params_prime} \rangle$	{CLOSING_PAR, COMMA}
$\langle \text{args_opt} \rangle$	{Arreglo, Booleano, CLOSING_PAR, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
$\langle \text{args_prime} \rangle$	{CLOSING_PAR, COMMA}
$\langle \text{array_args_opt} \rangle$	{Arreglo, Booleano, CLOSING_BRA, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
$\langle \text{array_args_prime} \rangle$	{CLOSING_BRA, COMMA}
$\langle \text{obj_elements} \rangle$	{CLOSING_KEY, IDENT}
$\langle \text{obj_tail} \rangle$	{COLON, OPENING_PAR}
$\langle \text{obj_prime} \rangle$	{CLOSING_KEY, COMMA}

3. Conjuntos FIRST

Símbolo	FIRST
$\langle \text{start} \rangle$	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{global_code} \rangle$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, ε , falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{block_code} \rangle$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, ε , falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{g_stmt} \rangle$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{g_decl_list} \rangle$	{IDENT}
$\langle \text{g_decl_init} \rangle$	{ASSIGN, ε }
$\langle \text{g_decl_list_prime} \rangle$	{COMMA, ε }
$\langle \text{g_semicolon_opt} \rangle$	{SEMICOLON, ε }
$\langle \text{g_sino_opt} \rangle$	{ ε , sino}
$\langle \text{g_sino_tail} \rangle$	{OPENING_KEY, si}
$\langle \text{b_stmt} \rangle$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
$\langle \text{b_decl_list} \rangle$	{IDENT}
$\langle \text{b_decl_init} \rangle$	{ASSIGN, ε }
$\langle \text{b_decl_list_prime} \rangle$	{COMMA, ε }
$\langle \text{b_semicolon_opt} \rangle$	{SEMICOLON, ε }
$\langle \text{b_sino_opt} \rangle$	{ ε , sino}
$\langle \text{b_sino_tail} \rangle$	{OPENING_KEY, si}
$\langle \text{decl_keyword} \rangle$	{const, mut, var}
$\langle \text{expr_or_crear} \rangle$	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, crear, falso, indefinido, nulo, verdadero}

Símbolo	FIRST
<code><crear_args_opt></code>	{OPENING_PAR, ϵ }
<code><crear_type></code>	{Arreglo, Cadena, IDENT, Matriz}
<code><para_init></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, const, ϵ , falso, indefinido, mut, nulo, var, verdadero}
<code><casos_list></code>	{caso, ϵ , porDefecto}
<code><block_stmt></code>	{OPENING_KEY}
<code><expr_opt></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, ϵ , falso, indefinido, nulo, verdadero}
<code><expr></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
<code><expr_prime></code>	{AND, DIV, EQUAL, GEQ, GREATER, LEQ, LESS, MINUS, MOD, NEQ, OR, PLUS, POWER, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, ϵ }
<code><expr_or_consola></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, consola, falso, indefinido, nulo, verdadero}
<code><arg_expr></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
<code><arg_expr_prime></code>	{AND, DIV, EQUAL, GEQ, GREATER, LEQ, LESS, MINUS, MOD, NEQ, OR, PLUS, POWER, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, ϵ }
<code><arg_expr_or_consola></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, consola, falso, indefinido, nulo, verdadero}
<code><ternary_mid_expr></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, consola, falso, indefinido, nulo, verdadero}
<code><ternary_mid_expr_prime></code>	{AND, DIV, EQUAL, GEQ, GREATER, LEQ, LESS, MINUS, MOD, NEQ, OR, PLUS, POWER, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, ϵ }
<code><consola_call></code>	{consola}
<code><consola_method></code>	{afirmar, agrupar, error, escribir, info, limpiar, tabla}
<code><factor></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
<code><factor_tail></code>	{ARROW, ASSIGN, DECREMENT, DIV_ASSIGN, INCREMENT, MINUS_ASSIGN, MOD_ASSIGN, OPENING_BRA, OPENING_PAR, PERIOD, PLUS_ASSIGN, POWER_ASSIGN, TIMES_ASSIGN, ϵ }
<code><arrow_body></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
<code><params_opt></code>	{IDENT, ϵ }
<code><params_prime></code>	{COMMA, ϵ }
<code><args_opt></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, ϵ , falso, indefinido, nulo, verdadero}
<code><args_prime></code>	{COMMA, ϵ }
<code><array_args_opt></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, ϵ , falso, indefinido, nulo, verdadero}
<code><array_args_prime></code>	{COMMA, ϵ }
<code><obj_elements></code>	{IDENT, ϵ }
<code><obj_tail></code>	{COLON, OPENING_PAR}
<code><obj_prime></code>	{COMMA, ϵ }

4. Conjuntos FOLLOW

Símbolo	FOLLOW
<code><start></code>	{EOF}
<code><global_code></code>	{EOF}
<code><block_code></code>	{CLOSING_KEY, caso, porDefecto}
<code><g_stmt></code>	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}

Símbolo	FOLLOW
<code><g_decl_list></code>	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
<code><g_decl_init></code>	{Arreglo, Booleano, COMMA, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
<code><g_decl_list_prime></code>	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
<code><g_semicolon_opt></code>	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
<code><g_sino_opt></code>	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
<code><g_sino_tail></code>	{Arreglo, Booleano, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, retornar, romper, si, var, verdadero}
<code><b_stmt></code>	{Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><b_decl_list></code>	{Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><b_decl_init></code>	{Arreglo, Booleano, CLOSING_KEY, COMMA, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><b_decl_list_prime></code>	{Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><b_semicolon_opt></code>	{Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><b_sino_opt></code>	{Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><b_sino_tail></code>	{Arreglo, Booleano, CLOSING_KEY, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><decl_keyword></code>	{IDENT}
<code><expr_or_crear></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><crear_args_opt></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><crear_type></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}

Símbolo	FOLLOW
<code><para_init></code>	{SEMICOLON}
<code><casos_list></code>	{CLOSING_KEY}
<code><block_stmt></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, capturar, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, sino, var, verdadero}
<code><expr_opt></code>	{Arreglo, Booleano, CLOSING_KEY, CLOSING_PAR, Cadena, EOF, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, SEMICOLON, STR, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><expr></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><expr_prime></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><expr_or_consola></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><arg_expr></code>	{CLOSING_PAR, COMMA}
<code><arg_expr_prime></code>	{CLOSING_PAR, COMMA}
<code><arg_expr_or_consola></code>	{CLOSING_PAR, COMMA}
<code><ternary_mid_expr></code>	{COLON}
<code><ternary_mid_expr_prime></code>	{COLON}
<code><consola_call></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><consola_method></code>	{OPENING_PAR}
<code><factor></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><factor_tail></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><arrow_body></code>	{AND, Arreglo, Booleano, CLOSING_BRA, CLOSING_KEY, CLOSING_PAR, COLON, COMMA, Cadena, DIV, EOF, EQUAL, GEQ, GREATER, IDENT, Infinito, LEQ, LESS, MINUS, MOD, Mate, Matriz, NEQ, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, OR, PLUS, POWER, SEMICOLON, STR, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES, caso, consola, const, continuar, elegir, falso, funcion, hacer, indefinido, intentar, mientras, mut, nulo, para, porDefecto, retornar, romper, si, var, verdadero}
<code><params_opt></code>	{CLOSING_PAR}
<code><params_prime></code>	{CLOSING_PAR}
<code><args_opt></code>	{CLOSING_PAR}
<code><args_prime></code>	{CLOSING_PAR}
<code><array_args_opt></code>	{CLOSING_BRA}
<code><array_args_prime></code>	{CLOSING_BRA}
<code><obj_elements></code>	{CLOSING_KEY}
<code><obj_tail></code>	{CLOSING_KEY, COMMA}
<code><obj_prime></code>	{CLOSING_KEY}

5. Conflictos

No terminal	Tokens en conflicto
<code><g_stmt></code>	{OPENING_KEY}
<code><g_semicolon_opt></code>	{SEMICOLÓN}
<code><b_stmt></code>	{OPENING_KEY}
<code><b_semicolon_opt></code>	{SEMICOLÓN}
<code><crear_args_opt></code>	{OPENING_PAR}
<code><expr_opt></code>	{Arreglo, Booleano, Cadena, IDENT, Infinito, MINUS, Mate, Matriz, NOT, NUMBER, NuN, Numero, OPENING_BRA, OPENING_KEY, OPENING_PAR, PLUS, STR, falso, indefinido, nulo, verdadero}
<code><expr_prime></code>	{AND, DIV, EQUAL, GEQ, GREATER, LEQ, LESS, MINUS, MOD, NEQ, OR, PLUS, POWER, STRICT_EQUAL, STRICT_NEQ, TERNARY, TIMES}
<code><factor_tail></code>	{OPENING_BRA, OPENING_PAR}
<code><arrow_body></code>	{OPENING_KEY}